

Data Structures & Algorithms

A Comprehensive Curated Reference Guide of Classic Problems and Solutions

1. Arrays & Hashing

Problem 1.1: Two Sum

Given an array of integers `nums` and an integer `target`, return indices of the two numbers such that they add up to `target`. You may assume that each input would have exactly one solution, and you may not use the same element twice.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
def twoSum(nums: list[int], target: int) -> list[int]:
    num_map = {} # val -> index
    for i, num in enumerate(nums):
        diff = target - num
        if diff in num_map:
            return [num_map[diff], i]
        num_map[num] = i
    return []
```

Problem 1.2: Valid Anagram

Given two strings `s` and `t`, return `true` if `t` is an anagram of `s`, and `false` otherwise.

Time Complexity: $O(n)$

Space Complexity: $O(1)$ (Assuming fixed alphabet size)

```
def isAnagram(s: str, t: str) -> bool:
    if len(s) != len(t):
        return False

    countS, countT = {}, {}
    for i in range(len(s)):
        countS[s[i]] = countS.get(s[i], 0) + 1
        countT[t[i]] = countT.get(t[i], 0) + 1

    return countS == countT
```

2. Two Pointers & Sliding Window

Problem 2.1: Best Time to Buy and Sell Stock

You are given an array `prices` where `prices[i]` is the price of a given stock on the `i`-th day. You want to maximize your profit by choosing a single day to buy one stock and choosing a different day in the future to sell that stock.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
def maxProfit(prices: list[int]) -> int:
    min_price = float('inf')
    max_prof = 0

    for price in prices:
        if price < min_price:
            min_price = price
        elif price - min_price > max_prof:
            max_prof = price - min_price

    return max_prof
```

3. Stacks & Queues

Problem 3.1: Valid Parentheses

Given a string *s* containing just the characters '(', ')', '{', '}', '[' and ']', determine if the input string is valid.

Time Complexity: $O(n)$

Space Complexity: $O(n)$

```
def isValid(s: str) -> bool:
    stack = []
    close_to_open = {")": "(", "}": "{", "]": "["}

    for c in s:
        if c in close_to_open:
            if stack and stack[-1] == close_to_open[c]:
                stack.pop()
            else:
                return False
        else:
            stack.append(c)

    return True if not stack else False
```

4. Linked Lists

Problem 4.1: Reverse Linked List

Given the head of a singly linked list, reverse the list, and return the reversed list.

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
def reverseList(head: ListNode) -> ListNode:
    prev, curr = None, head
    while curr:
        nxt = curr.next
        curr.next = prev
        prev = curr
        curr = nxt
    return prev
```

5. Trees & Graphs

Problem 5.1: Invert Binary Tree

Given the root of a binary tree, invert the tree, and return its root.

Time Complexity: $O(n)$

Space Complexity: $O(h)$ (where h is tree height)

```
def invertTree(root: TreeNode) -> TreeNode:
    if not root:
        return None

    # Swap the children
    root.left, root.right = root.right, root.left

    invertTree(root.left)
    invertTree(root.right)
    return root
```

6. Dynamic Programming

Problem 6.1: Climbing Stairs

You are climbing a staircase. It takes n steps to reach the top. Each time you can either climb 1 or 2 steps. In how many distinct ways can you climb to the top?

Time Complexity: $O(n)$

Space Complexity: $O(1)$

```
def climbStairs(n: int) -> int:
    one, two = 1, 1
    for i in range(n - 1):
        temp = one
        one = one + two
        two = temp
    return one
```